

IPPM Working Group
Internet-Draft
Intended status: Standards Track
Expires: 22 March 2027

R. Gandhi, Ed.
Cisco Systems, Inc.
T. Zhou
Huawei
Z. Li
China Mobile
W. Hawkins
University of Cincinnati
18 September 2026

Simple Two-Way Active Measurement Protocol (STAMP) Extensions for
Reflecting STAMP Packet IP Headers
draft-ietf-ippm-stamp-ext-hdr-14

Abstract

The Simple Two-Way Active Measurement Protocol (STAMP) and its optional extensions can be used for Edge-to-Edge (E2E) active measurements. In Situ Operations, Administration, and Maintenance (IOAM) data fields can be used for recording and collecting Hop-by-Hop (HBH) and E2E operational and telemetry information. This document extends STAMP to reflect IPv4/IPv6 headers as well as IPv6 extension headers for HBH and E2E active measurements, for example, using the IOAM data fields.

Commenté [MB1]: To make it explicit that it covers both address families.

This document specifies the requirements for IPv6 STAMP in

unauthenticated mode using UDP ~~zero-checksum~~ zero checksum, which deviates from the

integrity requirement in RFC 6936 (Applicability Statement for the Use of IPv6 UDP Datagrams with Zero Checksums).

Commenté [MB2]: I suggest to delete this part from the abstract. These details are repeated at least twice in the doc.

Status of This Memo

This Internet-Draft is submitted in full conformance with the provisions of BCP 78 and BCP 79.

Internet-Drafts are working documents of the Internet Engineering Task Force (IETF). Note that other groups may also distribute working documents as Internet-Drafts. The list of current Internet-Drafts is at <https://datatracker.ietf.org/drafts/current/>.

Internet-Drafts are draft documents valid for a maximum of six months and may be updated, replaced, or obsoleted by other documents at any time. It is inappropriate to use Internet-Drafts as reference material or to cite them other than as "work in progress."

This Internet-Draft will expire on 22 March 2027.

Copyright Notice

Copyright (c) 2026 IETF Trust and the persons identified as the

document authors. All rights reserved.

This document is subject to BCP 78 and the IETF Trust's Legal Provisions Relating to IETF Documents (<https://trustee.ietf.org/license-info>) in effect on the date of publication of this document. Please review these documents carefully, as they describe your rights and restrictions with respect to this document. Code Components extracted from this document must include Revised BSD License text as described in Section 4.e of the Trust Legal Provisions and are provided without warranty as described in the Revised BSD License.

Table of Contents

1. Introduction 3

2. Conventions Used in This Document 3

2.1. Requirements Language 4

2.2. Abbreviations 4

2.3. STAMP Reference Topology 5

3. Overview 5

3.1. IP/UDP Header 6

3.2. Procedure for Reflecting IPv6 Extension Headers 6

3.2.1. One-Way and Two-Way Measurement Types 10

3.3. Receiving IP Headers and IPv6 Extension Headers from the Data Plane 10

3.4. Procedure for Reflecting Fixed Headers 11

3.5. Reflecting Fixed Headers and IPv6 Extension Headers . . . 13

4. Use Case of Reflecting IOAM Data Fields 14

5. STAMP Extensions 15

5.1. Reflected IPv6 Extension Header Data TLV 15

5.2. Reflected Fixed Header Data TLV 16

5.3. IPv6 Extension Header Control Sub-TLV 18

6. UDP Checksum Handling 19

6.1. IPv4 UDP Zero-Checksum 20

6.2. IPv6 UDP Zero-Checksum 20

7. Operational Considerations 22

7.1. STAMP Session State Notification 23

7.2. Rate Limiting 23

8. Security Considerations 24

9. Implementation Status 24

10. IANA Considerations 25

11. References 26

11.1. Normative References 26

11.2. Informative References 27

Acknowledgments 29

Authors' Addresses 29

1. Introduction

The Simple Two-Way Active Measurement Protocol (STAMP) [RFC8762] provides capabilities for the measurement of various performance metrics in IP networks ~~[RFC8762]~~ without the use of a control channel to pre-signal session parameters. [RFC8972] defines optional extensions in the form of TLVs for STAMP. ~~STAMP-TestSTAMP-test~~ packets are transmitted along a path between a Session-Sender and a Session-Reflector to measure Edge-to-Edge (E2E) performance metrics, such as delay, delay variation, and

Commenté [MB3]: Please update through the doc for consistency with the base spec.

packet loss along that path.

In Situ Operations, Administration, and Maintenance (IOAM) is used for recording and collecting operational and telemetry information while ~~the~~ a packet traverses a path between two points in the network. ~~The IOAM data fields are defined in [RFC9197].~~ The information from the collected IOAM data fields [RFC9197] can be used to support Hop-by-Hop (HBH) and ~~Edge-to-Edge (E2E)~~ measurement use cases.

Commenté [MB4]: Consider citing a reference.

Commenté [MB5]: Simplify

Commenté [MB6]: Already introduced

IPv6 packets may carry IPv6 extension headers, including Hop-by-Hop Options headers and Destination Options headers, as defined in [RFC8200]. The HBH Options processing procedures are further specified in [RFC9673].

[RFC9486] specifies IPv6 option types for HBH and Destination Options headers to carry the IOAM Option-Types defined in [RFC9197] and [RFC9326] for the IPv6 data plane.

It may be desirable in some contexts (e.g., diagnostic) to record and collect HBH and E2E operational and telemetry information using active measurement packets between two nodes in a network. This is achieved by ~~augmenting STAMP [RFC8762]~~ using optional STAMP extensions defined in [RFC8972] to reflect IP headers as well as IPv6 extension headers as specified in this document. The procedure defined in this document leverages existing implementations at midpoint nodes with an IPv6 data plane that supports the IPv6 extension headers used, without any additional requirements.

This document specifies the requirements for IPv6 STAMP in unauthenticated mode using UDP zero-checksum, which deviates from the integrity requirement in [RFC6936]. Refer to Section XXX for more details.

Commenté [MB7]: Please complete.

2. Conventions Used in This Document

2.1. Requirements Language

The key words "MUST", "MUST NOT", "REQUIRED", "SHALL", "SHALL NOT", "SHOULD", "SHOULD NOT", "RECOMMENDED", "NOT RECOMMENDED", "MAY", and "OPTIONAL" in this document are to be interpreted as described in BCP 14 [RFC2119] [RFC8174] when, and only when, they appear in all capitals, as shown here.

2.2. Terminology

This document uses terms defined in [RFC8762], specifically Session-Sender, Session-Reflector, Session-Test packet, and symmetric.

This document uses terms defined in [RFC8972], specifically STAMP TLV Flags and Sub-TLVs.

2.3. Abbreviations

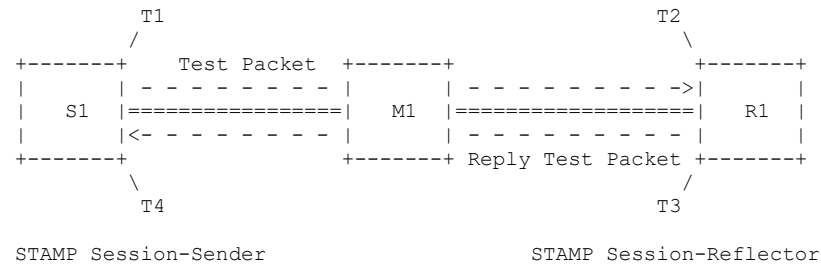
Abbreviation	Meaning	Reference
CRC	Cyclic Redundancy Check	[RFC6936]
CSPRNG	Cryptographically Secure Pseudorandom Number Generator	[NIST-CSPRNG]
DEX	Direct Export	[RFC9326]
E2E	Edge-to-Edge	[RFC9197]
HBH	Hop-by-Hop	[RFC8200]
HL	Hop Limit	[RFC8200]
IOAM	In Situ Operations, Administration, and Maintenance	[RFC9197]
MTU	Maximum Transmission Unit	[RFC8200]
STAMP	Simple Two-Way Active Measurement Protocol	[RFC8762]
TLV	Type-Length-Value	[RFC8972]
TTL	Time to Live	[RFC8200]
UDP	User Datagram Protocol	[RFC768]

Table 1: Abbreviations

2.3. STAMP Reference Topology

In the "STAMP Reference Topology" shown in Figure 1, the STAMP Session-Sender S1 initiates a Session-Sender test packet, and the STAMP Session-Reflector R1 transmits a reply Session-Reflector test packet. Node M1 is a midpoint node that does not perform any STAMP processing.

T1 is a transmit timestamp, and T4 is a receive timestamp added by node S1 in a STAMP test packet payload. T2 is a receive timestamp, and T3 is a transmit timestamp added by node R1 in a STAMP test packet payload.



Commenté [MB8]: Only used one. Suggest to expand it there and delete this entry

Commenté [MB9]: Not used in the doc. Suggest to delete this entry.

Commenté [MB10]: This is used twice. No need to have it here.

Commenté [MB11]: I would use «Hop Limit» through the doc and avoid having HL thing.

Commenté [MB12]: This is well-known. Suggest to delete.

Commenté [MB13]: Do we assume that M1 is always in both forward/reverse paths of a session?

Figure 1: STAMP Reference Topology

3. Overview

~~[RFC8972] defines optional extensions for STAMP. The optional extensions are added to the base STAMP test packet defined in [RFC8762] in the form of TLVs.~~ As specified in [RFC8972], both Session-Sender and Session-Reflector ~~STAMP-test-Test~~ packets are symmetric in size when including all optional TLVs (but excluding transport and network headers). ~~The A~~ Session-Reflector reflects all received STAMP TLVs from the Session-Sender ~~STAMP-test-Test~~ packet.

Commenté [MB14]: Do we really need to repeat this?
Already said above.

As specified in [RFC8762], ~~STAMP-STAMP-test-Test~~ packets are transmitted with IP/UDP headers. Since midpoint nodes do not process the UDP headers in the packets, they are agnostic to the ~~STAMP-STAMP-T~~est packets in the payload.

STAMP-~~t~~-Test packets may carry IP headers and IPv6 extension headers. This document defines procedures and STAMP extensions for a Session-Reflector to reflect the received IP headers and IPv6 extension headers back to the Session-Sender for ~~both one-way and two-way measurement types~~.

Commenté [MB15]: Add a reference to the section where this is defined

3.1. IP/UDP Header

The IP/UDP header specified in this section is applicable to all STAMP sessions using an IP/UDP header and is not limited to STAMP sessions using the extensions defined in this document.

The STAMP Session-Sender and Session-Reflector IP addresses for a STAMP session are provisioned on both ends of the STAMP session. ~~How such provisioning is achieved is out of scope.~~

The base ~~STAMP-STAMP-T~~est packet payloads can be transported using a UDP header with destination UDP port number 862 as the default destination port number, as specified in Section 4.1 of [RFC8762], or using a port number from the User Ports ~~(aka Registered Ports)~~ or Dynamic Ports ~~(aka Private or Ephemeral Ports)~~ ranges defined in Section 6 of [RFC6335].

Commenté [MB16]: Redundant with RFC6335

The source port number is chosen as follows:

- * The source UDP port number SHOULD be chosen using a randomized allocation method as specified in [RFC6056] to provide protection against off-path attacks, as recommended in [RFC8085].
- * The source UDP port number SHOULD be chosen from the Dynamic Ports range (49152-65535) [RFC6335] to avoid conflicts with well-known and registered service ports.
- * The source UDP port number MUST distinguish between the received

Session-Reflector test packets and the Session-Sender test packets from the reverse direction.

The IPv4 TTL and IPv6 Hop Limit MUST be set to 255 in both Session-Sender and Session-Reflector test packets, which allows determination of the number of hops that forwarded the test packet. Both the Session-Sender and the Session-Reflector MUST NOT discard the received Session-Sender STAMP test packets when the TTL or IPv6 Hop Limit is not 255.

Commenté [MB17]: ???

3.2. Procedure for Reflecting IPv6 Extension Headers

This document defines a new TLV option for STAMP, called "Reflected IPv6 Extension Header Data" (value TBA1). When a STAMP Session-Sender adds an IPv6 extension header, such as an IPv6 Hop-by-Hop Options header or a Destination Options header [RFC8200], in the Session-Sender test packet, the Session-Sender MUST add a corresponding "Reflected IPv6 Extension Header Data" TLV in the Session-Sender test packet to receive a copy of that IPv6 extension header back in the STAMP TLV. The Length of the TLV is set to the total size of the IPv6 extension header, starting from its Next Header field, and includes the first 8 octets of the IPv6 extension header carried in the Requested IPv6 Extension Header Data field.

Commenté [MB18]: Be explicit about which TLV

The format of this TLV is specified in Section 5.1.

Commenté [MB19]: I would introduce first the TLV format and then the procedures.

An example STAMP test packet for carrying an IPv6 header, IPv6 extension headers, and reflected data in the "Reflected IPv6 Extension Header Data" TLVs is shown in Figure 2.

That is, consider swapping the current sections order

```
+-----+
| IPv6 Header |
+-----+
| IPv6 Extension Header-1 RFC 8200 |
+-----+
~ ... ~
+-----+
| IPv6 Extension Header-N RFC 8200 |
+-----+
| UDP Header |
+-----+
| STAMP Packet RFC 8972 |
+-----+
| Reflected IPv6 Extension Header-1 Data STAMP TLV (TBA1) |
+-----+
~ ... ~
+-----+
| Reflected IPv6 Extension Header-M Data STAMP TLV (TBA1) |
+-----+
```

Commenté [MB20]: How to read this?

Note: Value of M <= N

Figure 2: Example Session-Sender and Session-Reflector Test Packet with Reflected IPv6 Extension Header Data TLVs

When adding multiple IPv6 extension headers in a Session-Sender test

packet, the corresponding "Reflected IPv6 Extension Header Data" TLVs MUST be added in the same order as the IPv6 extension headers to receive copies of those IPv6 extension headers. When ~~the a~~ Session-Sender test packets carry an IPv6 extension header that the Session-Sender does not require the Session-Reflector to reflect in Session-Reflector test packets, the Session-Sender MUST NOT add a corresponding "Reflected IPv6 Extension Header Data" TLV in the Session-Sender test packets. In this case, the number of "Reflected IPv6 Extension Header Data" TLVs (value of M in Figure 2) in the Session-Sender test packet would be less than the number of IPv6 extension headers (value of N in Figure 2).

The number of "Reflected IPv6 Extension Header Data" TLVs MUST be less than or equal to the number of IPv6 extension headers in a Session-Sender test packet.

The "Reflected IPv6 Extension Header Data" field in the "Reflected IPv6 Extension Header Data" TLV in Session-Sender test packets MUST be initialized to all zeros. When there is ambiguity because multiple IPv6 extension headers with the same length are present and not all of them need to be copied and reflected in the STAMP TLVs, the Session-Sender MUST populate the "Requested IPv6 Extension Header Data" field (shown in Figure 6) with the first 8 octets from the IPv6 extension header, starting from the Next Header field. This method

assumes that the first 8 octets of the IPv6 extension header do not change before being received at the Session-Reflector.

When the Session-Reflector receives a STAMP ~~t~~-Test packet with an IPv6 extension header and a "Reflected IPv6 Extension Header Data" TLV and the Session-Reflector supports this TLV, the following rules apply:

1. If the Session-Reflector receives a Session-Sender test packet with a non-zero "Requested IPv6 Extension Header Data" field in the "Reflected IPv6 Extension Header Data" TLV, it MUST copy the corresponding IPv6 extension header only if the requested length matches the extension-header length and the first 8 octets match, starting from the Next Header field of the IPv6 extension header.

If the "Requested IPv6 Extension Header Data" field is zero, the Session-Reflector MUST select the first IPv6 extension header with the matching length and copy its first 8 octets (starting from the Next Header field of the IPv6 extension header) in the "Requested IPv6 Extension Header Data" field of the "Reflected IPv6 Extension Header Data" TLV.

If no IPv6 extension header matches the requested data and length, the Session-Reflector MUST return the "Reflected IPv6 Extension Header Data" TLV with the Conformant Reflected Packet STAMP TLV flag C-flag ((Conformance)) set to 1 ~~in the STAMP TLV Flags~~, using the procedure defined in [I-D.ietf-ippm-asymmetrical-pkts].

2. The Session-Reflector MUST copy the portion of the IPv6 extension header after the first 8 octets into the "Reflected IPv6 Extension Header Data" field in the Session-Reflector test packet.

Commenté [MB21]: I guess this covers also cases where multiple instance of the same EH type is included?

If so, please say that in the text. Thanks

Commenté [MB22]: Please add this as a caution (as a new sub-section) in the Operational Considerations Section to better highlight this.

Commenté [MB23]: To be consistent with the asymmetric spec.

3. When there are multiple IPv6 extension headers in the received Session-Sender test packet, each IPv6 extension header MUST be processed in order, starting from the outer header, and copied into the corresponding "Reflected IPv6 Extension Header Data" TLV in the Session-Reflector test packet, if that STAMP TLV exists.

4. When the Session-Reflector receives a STAMP test packet with an IPv6 extension header but without a corresponding "Reflected IPv6 Extension Header Data" TLV, the Session-Reflector does not copy the IPv6 extension header into the Session-Reflector test packet.

The Session-Sender and Session-Reflector MUST ensure that the complete resulting test packet, including all applicable IP, UDP, STAMP, and reflected-header TLV fields, does not exceed the applicable path MTU after adding "Reflected IPv6 Extension Header Data" TLVs. If necessary, one or more "Reflected IPv6 Extension Header Data" TLVs MUST be removed to avoid violating the path MTU limit.

Because this procedure leverages existing IPv6 extension header implementations at midpoint nodes, no additional requirements are imposed when these headers are carried in STAMP test packets. IPv6 extension headers are processed by midpoint nodes using the procedures specified in their defining documents.

[RFC8250] precludes the insertion and deletion of IPv6 extension headers along the path (except by encapsulating the original packet in another IPv6 header); therefore, the use case where the IPv6 extension headers of the Session-Sender test packets are added, removed, or adjusted in length along the path is outside the scope of this document.

Examples of IPv6 extension headers include:

- * The IOAM data fields in an IPv6 Options header defined in [RFC9486].
- * The Performance and Diagnostic Metrics IPv6 Options header defined in [RFC8250].
- * The Maximum Path MTU IPv6 Options header defined in [RFC9268].
- * The Alternate Marking Method IPv6 Options header defined in [RFC9343].
- * The IPv6 Routing Header including the Segment Routing Header defined in [RFC8754].
- * Any new IPv6 extension header defined in the future.

3.2.1. One-Way and Two-Way Measurement Types

This document defines two measurement types: one-way and two-way measurements. These types relate only to whether the Session-Reflector adds new matching IPv6 extension headers for the reverse path.

Commenté [MB24]: Does it do so independent of the EH validation? Even if an EH is discarded?

Commenté [MB25]: MUST NOT?

Commenté [MB26]: Better to highlight this early in the document. E.g., a «Scope» section

Commenté [MB27]: What is purpose of having a list of EHs listed here?

Commenté [MB28]: Is this an example?

In the two-way measurement type, the Session-Reflector adds locally generated matching IPv6 extension headers in the Session-Reflector test packets in the same order as the IPv6 extension headers received in the Session-Sender test packets for the reverse-direction measurement. These headers need not be byte-for-byte copies of the received headers; their length and content are local decisions at the Session-Reflector. Routing headers specific to the Session-Sender are not required to be inserted into the reverse-direction Session-Reflector test packet. The STAMP Session-Sender enables this type by adding the "IPv6 Extension Header Control" Sub-TLV ~~(Section XX)~~ for the "Reflected Test Packet Control" TLV (Type 12) [\[I-D.ietf-ippm-asymmetrical-pkts\]](#) in the Session-Sender test packets.

Commenté [MB29]: Please complete

In the one-way measurement type, ~~the~~^a Session-Reflector MAY omit adding new matching IPv6 extension headers in the Session-Reflector test packets in response to the received IPv6 extension headers in the Session-Sender test packets, based on its local policy. This does not preclude the Session-Reflector from adding IPv6 extension headers independently. However, the Session-Reflector still copies received IPv6 extension headers into the "Reflected IPv6 Extension Header Data" TLVs ~~as specified in Section 3.2.~~ ^{This type is the default if the "IPv6 Extension Header Control" Sub-TLV is absent in the Session-Sender test packet.}

Commenté [MB30]: This is cyclic as we are in 3.2.1 and pointing to 3.2. Please fix.

The measurement type for a STAMP session is locally provisioned on the STAMP Session-Sender.

Commenté [MB31]: Can we please have a summary of requirement provisioning tasks as a brief sub-section of the OPS Considerations section?

3.3. Receiving IP Headers and IPv6 Extension Headers from the Data Plane

~~The STAMP-Stamp-test-Test~~ packets carrying the IP header and IPv6 extension headers for HBH and E2E measurements are received by ~~the~~^{an} egress node hosting the Session-Reflector. When the received STAMP test packets are processed by the data plane on the egress node that has the Session-Reflector, ~~the data plane MUST provide the received IP header and IPv6 extension headers from the Session-Sender test packets to the Session-Reflector on the node.~~

Commenté [MB32]: This is an internal interface, but from an OPS standpoint, we need to check whether this is supported (as part of verify correct operation). No need to specify how, but please add this under OPS Cons Section.

This applies for both sender/receiver.

You may indicate that operators may also use rfc9740 (informative, only) to retrieve EH, limits, etc. and complement the procedure in this document.

Similarly, when the received STAMP test packets are processed by the data plane in the reverse direction on the ingress node that has the Session-Sender, the data plane MUST provide the received IP header and IPv6 extension headers from the Session-Reflector test packets to the Session-Sender on the node for the two-way measurement type.

3.4. Procedure for Reflecting Fixed Headers

This document defines a new TLV option for STAMP, called "Reflected Fixed Header Data" (value TBA2). ~~The This~~ STAMP TLV can be used to reflect any fixed-size header received in a Session-Sender test packet, including IPv4 and IPv6 headers. When a STAMP Session-Sender adds an IP header, the Session-Sender also adds a "Reflected Fixed Header Data" TLV in the Session-Sender test packet to receive a copy of that IP header back in the STAMP TLV. The Length of the TLV is set to the total size of the IP header, and includes the first 4

octets of the target IP header carried in the Requested Fixed Header Data field. The format of this TLV is specified in Section 5.2.

An example STAMP test packet carrying an IP header and reflected data in the "Reflected Fixed Header Data" TLV is shown in Figure 3.

```
+-----+
| IP Header                                     |
+-----+
| UDP Header                                   |
+-----+
| STAMP Packet RFC 8972                       |
+-----+
| Reflected Fixed Header Data STAMP TLV (TBA2) |
+-----+
```

Figure 3: Example Session-Sender and Session-Reflector Test Packet with "Reflected Fixed Header Data" TLV

When adding multiple IP headers in a Session-Sender test packet, the corresponding "Reflected Fixed Header Data" TLVs MUST also be added in the same order as the IP headers to receive copies of those IP headers. When the Session-Sender test packets carry an IP header that the Session-Sender does not require the Session-Reflector to reflect in Session-Reflector test packets, the Session-Sender MUST NOT add a corresponding "Reflected Fixed Header Data" TLV in the Session-Sender test packets. In this case, the number of "Reflected Fixed Header Data" TLVs in the Session-Sender test packet would be less than the number of IP headers in the packet.

The number of "Reflected Fixed Header Data" TLVs MUST be less than or equal to the number of IP headers in the Session-Sender test packet.

The "Reflected Fixed Header Data" field in the "Reflected Fixed Header Data" TLV in Session-Sender test packets MUST be initialized to all zeros. When there is ambiguity because due to the presence of multiple IP headers

with the same length ~~are present~~ and not all of them need to be copied and reflected in the STAMP TLVs, the Session-Sender MUST populate the "Requested Fixed Header Data" field (shown in Figure 7) with the first 4 octets from the IP header. This method assumes that the first 4 octets in the IP header do not change before being received at the Session-Reflector.

When the Session-Reflector receives a STAMP test packet with an IP header and a "Reflected Fixed Header Data" TLV and the Session-Reflector supports this TLV, the following rules apply:

1. If the Session-Reflector receives a Session-Sender test packet with a non-zero "Requested Fixed Header Data" field in the "Reflected Fixed Header Data" TLV, it MUST copy the corresponding IP header only if the requested length matches the IP-header length and the first 4 octets match.

If the "Requested Fixed Header Data" field is zero, the Session-Reflector MUST select the first IP header with the matching length and copy its first 4 octets in the "Requested Fixed Header Data" field of the "Reflected Fixed Header Data" TLV.

If no IP header matches the requested data and length, the Session-Reflector MUST return the "Reflected Fixed Header Data" TLV with the Conformant Reflected Packet STAMP TLV flagC-flag ~~(Conformance)~~ set to 1 ~~in the STAMP TLV~~ ~~Flags~~, using the procedure defined in [I-D.ietf-ippm-asymmetrical-pkts].

2. The Session-Reflector MUST copy the portion of the IP header after the first 4 octets into the "Reflected Fixed Header Data" field in the Session-Reflector test packet.
3. When there are multiple IP headers in the received Session-Sender test packet, each IP header MUST be processed in order, starting from the outer header, and copied into the corresponding "Reflected Fixed Header Data" TLV in the Session-Reflector test packet, if that STAMP TLV exists.
4. When the Session-Reflector receives a STAMP test packet with an IP header but without a corresponding "Reflected Fixed Header Data" TLV, the Session-Reflector does not copy the IP header into the Session-Reflector test packet.

The Session-Sender and Session-Reflector MUST ensure that the complete resulting test packet, including all applicable IP, UDP, STAMP, and reflected-header TLV fields, does not exceed the applicable path MTU after adding "Reflected Fixed Header Data" TLVs. If necessary, one or more "Reflected Fixed Header Data" TLVs MUST be removed to avoid violating the path MTU limit.

3.5. Reflecting Fixed Headers and IPv6 Extension Headers

STAMP test packets can be used to reflect both IP headers and IPv6 extension headers by carrying the corresponding "Reflected Fixed Header Data" (Section XX) and "Reflected IPv6 Extension Header Data" (Section XX) TLVs. An example of STAMP test packet carrying an IPv6 header and an IPv6 extension header along with their corresponding "Reflected Fixed Header Data" and "Reflected IPv6 Extension Header Data" TLVs is shown in Figure 4.

```
+-----+
| IPv6 Header |
+-----+
| IPv6 Extension Header RFC 8200 |
+-----+
| UDP Header |
+-----+
| STAMP Packet RFC 8972 |
+-----+
| Reflected Fixed Header Data STAMP TLV (TBA2) |
+-----+
| Reflected IPv6 Extension Header Data STAMP TLV (TBA1) |
+-----+
```

Figure 4: Example Session-Sender and Session-Reflector Test Packet with "Reflected Fixed Header Data" and "Reflected IPv6

Commenté [MB33]: How to interpret this mention?

Extension Header Data" TLVs

The "Reflected Fixed Header Data" TLVs MUST be added before adding the "Reflected IPv6 Extension Header Data" TLVs to maintain the same order as the ordered sequence of IP headers and IPv6 extension headers in the Session-Sender test packets. This ordering requirement and its validation apply independently of whether the corresponding headers are added to the reverse Session-Reflector test packet. If the "Reflected Fixed Header Data" TLVs and the "Reflected IPv6 Extension Header Data" TLVs are not received in this order, the Session-Reflector MUST return these TLVs with the Conformant Reflected Packet STAMP TLV flag ~~flag~~ (Conformance) set to 1 ~~in the STAMP TLV Flags~~ using the procedure defined in [I-D.ietf-ippm-asymmetrical-pkts], but without copying any data in these STAMP TLVs.

4. Use Case of Reflecting IOAM Data Fields

~~In-Situ Operations, Administration, and Maintenance~~ (IOAM) is used for recording and collecting operational and telemetry information while the packet traverses a path between two points in the network. The IOAM data fields are defined in [RFC9197]. Examples of data recorded by IOAM Trace Options include per-hop information, such as node ID, timestamp, queue depth, interface identifier~~ID~~, and interface load.

The information collected can be used for monitoring paths, proof-of-transit, and troubleshooting failures in the network. IOAM can be used with STAMP test packets for active measurements. The procedure and STAMP extensions defined in this document can be used to reflect the collected IOAM data fields back to the Session-Sender, where the Session-Sender can use this information to support HBH and E2E measurement use cases.

[RFC9486] defines types for HBH and Destination Options headers and is used to carry the IOAM option types defined in [RFC9197] for the IPv6 data plane. The STAMP Session-Sender and Session-Reflector test packets carry the IPv6 Options headers with IOAM option types for recording and collecting HBH and E2E operational and telemetry information for active measurements, as shown in Figure 5. The Session-Sender node, midpoint nodes, and the Session-Reflector node process the IOAM data fields, as defined in [RFC9197]. Note that using the IOAM option type "Incremental Trace Option-Type" is not supported by [RFC9486].

```
+-----+
| IPv6 Header                                     |
+-----+
| HBH IOAM IPv6 Options Header RFC 9486         |
+-----+
| UDP Header                                     |
+-----+
| STAMP Packet RFC 8972                         |
+-----+
| Reflected IPv6 Extension Header Data STAMP TLV (TBA1) |
+-----+
```

Figure 5: Example Session-Sender and Session-Reflector Test Packet for IOAM with Reflected IPv6 Extension Header Data TLV

IOAM Direct Exporting (DEX) [RFC9326] is applicable with STAMP test packets for on-path telemetry use cases as described in [I-D.ietf-ippm-on-path-active-measurements]. In this case, the Session-Reflector is not required to reflect the IOAM option type, since no IOAM data fields would be recorded in the STAMP test packets. Hence, the Session-Sender MAY omit a corresponding "Reflected IPv6 Extension Header Data" TLV in Session-Sender test packets for the IOAM DEX option type.

5. STAMP Extensions

5.1. Reflected IPv6 Extension Header Data TLV

The "Reflected IPv6 Extension Header Data" TLV is carried by Session-Sender and Session-Reflector test packets. STAMP test packets MAY carry one or more STAMP TLVs of this type. The same "Reflected IPv6 Extension Header Data" TLV Type is used for reflecting different IPv6 extension headers, including HBH and Destination IPv6 options headers. The format of the "Reflected IPv6 Extension Header Data" TLV is shown in Figure 6.

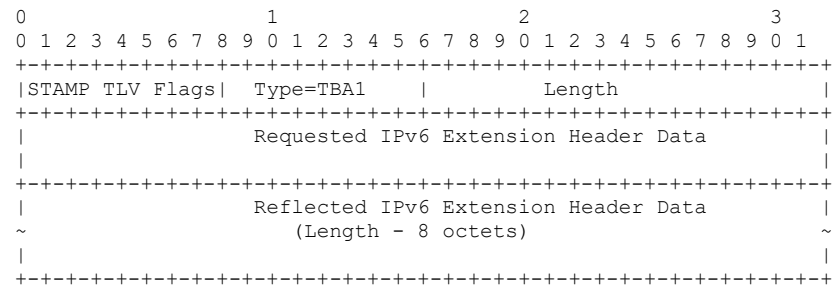


Figure 6: Reflected IPv6 Extension Header Data TLV

The Reflected IPv6 Extension Header Data TLV STAMP TLV fields are defined as follows:

- * Type: STAMP TLV Type (value TBA1).
- * STAMP TLV Flags: The STAMP TLV Flags follow the procedures described in [RFC8972].
- * Type: MUST be set to TBA1.
- * Length: A two-octet field equal to the total size of the IPv6 extension header to be reflected, starting from its Next Header field, in octets.
- * Requested IPv6 Extension Header Data: A fixed 8-octet field containing the first 8 octets of the target IPv6 extension header to be reflected, starting from its Next Header field.

This field is used to disambiguate which IPv6 extension header in the received Session-Sender test packet MUST be copied into the

Commenté [MB34]: 9326 is normative for this MAY

Alternatively, as this is only a use case example, s/MAY/may?

Commenté [MB35]: This section should be provided before the procedure as it is difficult to digest with the current order.

Commenté [MB36]: Follow the order as in the figure

Commenté [MB37]: Please add the section where this is defined.

~~* Type: STAMP TLV Type (value TBA2).~~

* STAMP TLV Flags: The STAMP TLV Flags follow the procedures described in [RFC8972].

* Type: MUST be set to TBA2.

* Length: A two-octet field equal to the total size of the IP header to be reflected, in octets. For a 20-octet IPv4 header, it is 20, and for a 40-octet IPv6 header, it is 40.

* Requested Fixed Header Data: A fixed 4-octet field containing the first 4 octets of the target IP header to be reflected.

This field is used to disambiguate which IP header in the received Session-Sender test packet MUST be copied into the Reflected field when multiple IP headers of the same length are present in the Session-Sender test packet.

When this field is set to all zeros, the Session-Reflector MUST match the first IP header in the Session-Sender test packet with the matching length.

* Reflected Fixed Header Data: A variable-length field of (Length - 4) octets containing the portion of the reflected IP header after the first 4 octets, copied from the received Session-Sender test packet by the Session-Reflector.

In Session-Sender test packets, this field MUST be initialized to zero.

When the Session-Reflector recognizes the received "Reflected Fixed Header Data" TLV but cannot use it to reflect any received IP header, the Session-Reflector MUST return the "Reflected Fixed Header Data" TLV with the Conformant Reflected Packet STAMP TLV flag

~~(Conformance)~~ set to 1 ~~in the STAMP TLV Flags~~ using the procedure defined in [I-D.ietf-ippm-asymmetrical-pkts]. This can occur, for example, if:

* There is a mismatch between the requested length and the received IP-header length.

* The Session-Reflector cannot access the received IP headers from the data plane or the data plane does not support the IP header.

* No IP header matches the requested data and length.

5.3. IPv6 Extension Header Control Sub-TLV

This document defines the "IPv6 Extension Header Control" Sub-TLV (Type TBA3) for the "Reflected Test Packet Control" TLV (Type 12) introduced in [I-D.ietf-ippm-asymmetrical-pkts]. The format of "IPv6 Extension Header Control" Sub-TLV is shown in Figure 8.

```

0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1
+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+
| Sub-TLV Flags | Type = TBA3 | Sub-TLV Length = 0 |
+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+

```

Figure 8: IPv6 Extension Header Control Sub-TLV

The IPv6 Extension Header Control Sub-TLV fields are defined as follows:

* Sub-TLV Flags: An eight-bit field. The format, values, and interpretation of flags are as defined for STAMP TLV Flags [RFC8972]. Flag values are taken from the "STAMP TLV Flags" registry [IANA-STAMP].
 * Type: Sub-TLV Type (value TBA3).

~~* Sub-TLV Flags: The Sub-TLV Flags follow the procedures for STAMP TLV Flags described in [RFC8972].~~

* Sub-TLV Length: A two-octet field equal to the length of the Data field in octets. It ~~is~~ MUST be set to 0.

When the Session-Reflector receives a STAMP test packet with the "IPv6 Extension Header Control" Sub-TLV and the Session-Reflector supports this Sub-TLV, the following rules apply:

1. The Session-Reflector MUST add locally generated matching IPv6 extension headers, subject to the relevant IPv6 data-plane specifications, in the same order as the IPv6 extension headers received in the Session-Sender test packet. The generated headers need not be byte-for-byte copies of the received headers, and routing headers specific to the Session-Sender are excluded from this requirement.
2. In the absence of the "IPv6 Extension Header Control" Sub-TLV in the received Session-Sender test packet, the Session-Reflector MAY omit adding locally generated matching IPv6 extension headers in the Session-Reflector test packet corresponding to the received IPv6 extension headers, based on its local policy. This does not preclude the Session-Reflector from adding IPv6 extension headers independently of the received IPv6 extension headers.
3. The IPv6 extension headers received in the Session-Sender test packets MUST be copied and reflected in the corresponding "Reflected IPv6 Extension Header Data" TLVs to the Session-Sender regardless of whether the "IPv6 Extension Header Control" Sub-TLV is present.
4. If the Session-Reflector cannot add a new matching IPv6 extension header in the Session-Reflector test packet, the Session-Reflector MUST return the "Reflected Test Packet Control" TLV (Type 12) with the Conformant Reflected Packet STAMP TLV flag ~~flag (Conformance)~~ set to 1 in the Sub-TLV Flags of the "IPv6 Extension Header Control" Sub-TLV using the procedure defined in [I-D.ietf-ippm-asymmetrical-pkts]. This can occur, for example, when the Session-Reflector does not support the IPv6 extension header or when the Session-Reflector cannot access the received IPv6 extension headers from the data plane.

Commenté [MB39]: Add as a normative reference

[IANA-STAMP]IANA, "STAMP Sub-TLV Types", <<https://www.iana.org/assignments/stamp-tlv-types>>.

STAMP test packets MUST NOT carry more than one "IPv6 Extension Header Control" Sub-TLV in a "Reflected Test Packet Control" TLV (Type 12). If the "Reflected Test Packet Control" TLV (Type 12) in the Session-Sender test packet contains more than one "IPv6 Extension Header Control" Sub-TLV, the Session-Reflector MUST return the "Reflected Test Packet Control" TLV (Type 12) with the Conformant Reflected Packet STAMP TLV flag ~~flag~~ ~~(Conformance)~~ set to 1 in the Sub-TLV Flags of each "IPv6 Extension Header Control" Sub-TLV, using the procedure defined in [I-D.ietf-ippm-asymmetrical-pkts].

6. UDP Checksum Handling

The UDP checksum handling specified in this section is applicable to all STAMP sessions using IPv4/UDP and IPv6/UDP and is not limited to STAMP sessions using the extensions defined in this document.

As specified in [RFC8085], the UDP checksum provides a statistical guarantee that the payload was not corrupted in transit, truncated, or padded.

The following example limitations for STAMP timestamping necessitate the exceptions to permit the use of UDP zero-checksum for IPv4 and IPv6 ~~;~~ :

- * When the local processor cannot recompute the UDP checksum after adding the timestamp in the STAMP test packet.
- * When the local processor cannot add a checksum complement [RFC7820] after adding the timestamp in the STAMP test packet.

6.1. IPv4 UDP Zero-Checksum

Use of the UDP checksum with IPv4 MUST be the default configuration for all implementations.

For IPv4, [RFC768] permits an option to disable UDP checksum processing by setting the checksum value to zero.

For IPv4 STAMP test packets, the Session-Sender and Session-Reflector can use this exception for the UDP port numbers specifically used in STAMP sessions to set the UDP checksum value to 0 with additional checks on the source and destination IP addresses in the STAMP test packets.

6.2. IPv6 UDP Zero-Checksum

~~STAMP test packets are the innermost payload and are not a tunnel encapsulation.~~ This document applies the exception in Section 8.1 of [RFC8200] to IPv6 STAMP even though STAMP is not a tunnel protocol, because the STAMP payload is the innermost protocol payload and has no inner packet whose integrity would be protected by a UDP checksum. The UDP zero-checksum requirements of [RFC6936] therefore apply directly to the STAMP payload rather than to a tunnel encapsulation carrying an inner packet.

For IPv6, any node that implements UDP zero-checksum mode MUST follow

the requirements specified in [RFC6936] and [RFC8085] as described below, with one deviation.

~~* In this specification, the use of UDP zero-checksum for IPv6 STAMP in unauthenticated mode deviates from requirement 5 in Section 5 of [RFC6936].~~

~~* Requirement 5 of Section 5 of [RFC6936] cannot be satisfied because STAMP is not a tunnel protocol and does not include an inner packet with a CRC or other mechanism for checking packet integrity in unauthenticated mode.~~

~~* This deviation from [RFC6936] requirement is limited to the IPv6 STAMP sessions in unauthenticated mode that MUST operate under the constraints listed below.~~

IPv6 UDP zero-checksum can be enabled only when the following requirements, recommendations, and constraints are satisfied and the residual risk due to STAMP test packet corruption is

~~acceptable: acceptable.~~

1. UDP zero-checksum is enabled only for the specific UDP port number or port range used by a STAMP session, at both the Session-Sender and Session-Reflector.

This corresponds to requirement 1 in Section 5 of [RFC6936].

2. STAMP test packets in authenticated mode, as defined in ~~Figures~~ Section 3 of [RFC8972] and Section 4 of [RFC8972], are RECOMMENDED in networks where packet integrity is required.

This corresponds to requirement 2 in Section 5 of [RFC6936].

3. Requirement 3 in Section 5 of [RFC6936] does not apply to STAMP because STAMP packets are not tunnel payloads and do not rely on an inner packet integrity check.

4. UDP zero-checksum is handled in STAMP so that corruption of header information is detected in STAMP and does not result in accumulation of incorrect state for the protocol.

This corresponds to requirement 4 in Section 5 of [RFC6936].

5. Requirements 6 and 7 in Section 5 of [RFC6936] are not applicable to STAMP as they are related to ~~the~~ keep alive messages.

6. Middleboxes within the controlled domain that process ~~the~~ IPv6 STAMP test packets MUST comply with Requirements-requirements 8 through 10 of Section 5 of [RFC6936].

Additional specific guidance from Section 3.4.1 of [RFC8085] applied to IPv6 STAMP test packets that use UDP zero-checksum is summarized below:

1. Use of the UDP checksum with IPv6 MUST be the default configuration for all implementations.

Commenté [MB40]: For whom? Operator?

Please say so.

This corresponds to the first requirement in Section 3.4.1 of [RFC8085].

2. The receiving endpoint MUST verify a non-zero UDP checksum packet and MUST discard it if checksum verification fails; it MUST NOT treat the packet as a valid measurement result.

This corresponds to the second requirement in Section 3.4.1 of [RFC8085] and also applies to Section 4 of [RFC6936] and Section 5 of [RFC6936].

3. The receiving endpoint MUST only permit the use of UDP zero-checksum for IPv6 on a UDP destination port number that is specifically enabled for STAMP and MUST check that the source and destination IPv6 addresses are valid and discard any packet for which this check fails.

This corresponds to the third requirement in Section 3.4.1 of [RFC8085].

4. STAMP sessions are restricted to networks under a single administrative domain (see Section 8), where the operator is willing to take the risk of STAMP test packet corruption affecting measurements when using UDP zero-checksum.

This corresponds to the fourth requirement in Section 3.4.1 of [RFC8085].

5. STAMP sessions that choose to use a UDP zero-checksum MUST NOT make assumptions regarding the correctness of received test packets and MUST behave correctly when a UDP datagram is corrupted.

This corresponds to the fifth requirement in Section 3.4.1 of [RFC8085].

7. Operational Considerations

The operational considerations specified in [RFC8762] and [I-D.ietf-ipbm-asymmetrical-pkts] apply to the procedure and extensions defined in this document.

In addition, the Management and Deployment Considerations specified in [RFC9197] also apply when using the IOAM data fields defined in that document.

An operator MAY provision a local policy on a Session-Reflector to not copy and reflect the received IPv6 extension headers and IP headers in the Session-Reflector test packets to avoid exposing the collected network information to ~~the-Session-Senders~~Session-Sender.

7.1. STAMP Session State Notification

The STAMP session state change notifications specified in this section are applicable to all STAMP sessions and are not limited to

STAMP sessions using the extensions defined in this document.

The STAMP session state monitoring allows the Session-Sender to determine whether the STAMP test is idle, active, or failed. A STAMP implementation **SHOULD** generate state-change notifications as follows:

- * The STAMP session state is reported as idle when the Session-Sender is not transmitting test packets.
- * The STAMP session state is initially reported as active when the Session-Sender is transmitting test packets and at least one Session-Reflector test packet is received.
- * The STAMP session state is reported as failed when N consecutive Session-Reflector test packets are not received after the STAMP session state is reported as active, where N (the consecutive packet loss count) is a locally provisioned value.
- * The STAMP session state transitions from failed back to active when the Session-Sender is transmitting test packets and at least one Session-Reflector test packet is again received.

Because STAMP test packets are transmitted over the path being measured, a connectivity failure of that path typically manifests as the continuous packet loss specified above, resulting in the STAMP session state being reported as failed.

7.2. Rate Limiting

The rate limiting considerations specified in this section are applicable to all STAMP sessions and are not limited to STAMP sessions using the extensions defined in this document.

On both Session-Sender and Session-Reflector nodes, as each STAMP test packet is processed by the control plane and consumes CPU and memory resources, it is subject to rate limiting as a protection against denial-of-service attacks. Such rate limiting on the punt path is indistinguishable from the actual loss in the network and can, therefore, be reported as packet loss.

It is useful for an operator to know that rate limiting was applied to STAMP test packets (for example, based on the UDP ports used for STAMP), so that the alerting system can correlate STAMP packets being rate-limited with failure notifications.

This throttling or policing of incoming STAMP test packets **SHOULD NOT** be more stringent than the transmit rate of the STAMP test packets to prevent invalid measurement results.

8. Security Considerations

The security considerations specified in [RFC8762], [RFC8972], [RFC8200], and [I-D.ietf-ippm-asymmetrical-pkts] apply to the procedure and extensions defined in this document. In addition, the security considerations specified in [RFC9197] and [RFC9486] also apply when using IPv6 options for IOAM data fields.

The procedures defined in this document are intended for deployment

Commenté [MB41]: Why isn't MUST?

IF SHOULD is kept, then please add some text about the implications of not following the SHOULD.

Commenté [MB42]: When this is allowed? Please clarify that in the text.

If we don't have an exception, then this need to be changed to MUST NOT.

in a single network administrative domain. It is assumed that the operator has verified the integrity of the forward and return paths used to transmit STAMP test packets so that collected network information is not exposed on an undesired node.

If desired, attacks can be mitigated by performing basic validation checks of the timestamp fields in received reply test packets at the Session-Sender. For example, verifying that T2 is later than T1 in the STAMP Reference Topology shown in Figure 1 requires clock synchronization between the Session-Sender and Session-Reflector. In contrast, checking that T3 is greater than or equal to T2, or that T4 is later than T1, compares timestamps generated at the same node and does not require clock synchronization. The minimal state associated with this protocol also limits the extent of measurement disruption that can be caused by a corrupt or invalid test packet to a single test cycle.

Furthermore, implementations SHOULD NOT assign STAMP Session-IDs [RFC8972] in a predictable manner. To avoid predictability, implementations can leverage a Cryptographically Secure Pseudorandom Number Generator [NIST-CSPRNG].

9. Implementation Status

Editorial note: Please remove this section prior to publication.

An open-source implementation of the Simple Two-Way Active Measurement Protocol [RFC8762] is available in Teaparty.

<https://github.com/cerfcast/teaparty>

An implementation of the solution in this document is available at the following location:

<https://github.com/cerfcast/teaparty/commit/393abf9357a6c2439877d9bcf2dc426dd89c7158>

The implemented features are as follows:

1. Extraction of the extension headers from the IPv6 headers of the received STAMP test packet.
2. Reflection of the extension headers in the reflected STAMP TLV data (with checks for matching length).
3. Adding the extension headers to the IP header of the reflected STAMP test packet.
4. Support for multiple IPv6 extension headers.
5. Reflection of the fixed IP header in the reflected STAMP TLV data.

There is also support for the reflected IPv6 extension header TLV data in the Wireshark dissector:

<https://github.com/cerfcast/teaparty/commit/fb74e2e02396e9bb3ead017e8d9a0c187e3573e2>

There is also support for tools to test the reflected IPv6 extension header TLV data:

https://github.com/cerfcast/teaparty/tree/main/testing_data#testing-reflected-ipv6-extension-header-data

Contact:

William Hawkins

University of Cincinnati

Email: hawkinsw@obs.cr

10. IANA Considerations

~~IANA has created the "STAMP TLV Types" registry for [RFC8972]. IANA is requested to allocate values for the "Reflected IPv6 Extension Header Data" TLV Type and the "Reflected Fixed Header Data" TLV Type the following TLVs from the IETF Review TLV range of the same "STAMP TLV Types" registry under the Simple Two-way Active Measurement Protocol (STAMP) TLV Types registry group:-~~

Value	Description	Reference
TBA1	Reflected IPv6 Extension Header Data	This document
TBA2	Reflected Fixed Header Data	This document

Table 2: STAMP TLV Types

~~IANA is requested to allocate a value for the Sub-TLV Type "IPv6 Extension Header Control" (Type TBA3) the following sub-TLV for the STAMP TLV Type "Reflected Test Packet Control" (Type 12) defined in [I-D.ietf-ippm-asymmetrical-pkts], from the "STAMP Sub-TLV Types" Registry under the Simple Two-way Active Measurement Protocol (STAMP) TLV Types registry group:-~~

Value	Description	TLV Used	Reference
TBA3	IPv6 Extension Header Control	Reflected Test Packet Control	This document

Table 3: Sub-TLV Type for Reflected Test Packet Control TLV

Note to the RFC Editor: Please replace all occurrences of TBA1, TBA2, and TBA3 with the values assigned by IANA.

11. References

a mis en forme : Anglais (États-Unis)
a mis en forme : Anglais (États-Unis)
a mis en forme : Anglais (États-Unis)

11.1. Normative References

- [RFC2119] Bradner, S., "Key words for use in RFCs to Indicate Requirement Levels", BCP 14, RFC 2119, DOI 10.17487/RFC2119, March 1997, <<https://www.rfc-editor.org/info/rfc2119>>.
- [RFC8174] Leiba, B., "Ambiguity of Uppercase vs Lowercase in RFC 2119 Key Words", BCP 14, RFC 8174, DOI 10.17487/RFC8174, May 2017, <<https://www.rfc-editor.org/info/rfc8174>>.
- [RFC8200] Deering, S. and R. Hinden, "Internet Protocol, Version 6 (IPv6) Specification", STD 86, RFC 8200, DOI 10.17487/RFC8200, July 2017, <<https://www.rfc-editor.org/info/rfc8200>>.
- [RFC8762] Mirsky, G., Jun, G., Nydell, H., and R. Foote, "Simple Two-Way Active Measurement Protocol", RFC 8762, DOI 10.17487/RFC8762, March 2020, <<https://www.rfc-editor.org/info/rfc8762>>.
- [RFC8972] Mirsky, G., Min, X., Nydell, H., Foote, R., Masputra, A., and E. Ruffini, "Simple Two-Way Active Measurement Protocol Optional Extensions", RFC 8972, DOI 10.17487/RFC8972, January 2021, <<https://www.rfc-editor.org/info/rfc8972>>.
- [RFC768] Postel, J., "User Datagram Protocol", STD 6, RFC 768, DOI 10.17487/RFC768, August 1980, <<https://www.rfc-editor.org/info/rfc768>>.
- [RFC6056] Larsen, M. and F. Gont, "Recommendations for Transport-Protocol Port Randomization", BCP 156, RFC 6056, DOI 10.17487/RFC6056, January 2011, <<https://www.rfc-editor.org/info/rfc6056>>.
- [RFC6335] Cotton, M., Eggert, L., Touch, J., Westerlund, M., and S. Cheshire, "Internet Assigned Numbers Authority (IANA) Procedures for the Management of the Service Name and Transport Protocol Port Number Registry", BCP 165, RFC 6335, DOI 10.17487/RFC6335, August 2011, <<https://www.rfc-editor.org/info/rfc6335>>.
- [RFC6936] Fairhurst, G. and M. Westerlund, "Applicability Statement for the Use of IPv6 UDP Datagrams with Zero Checksums", RFC 6936, DOI 10.17487/RFC6936, April 2013, <<https://www.rfc-editor.org/info/rfc6936>>.
- [RFC8085] Eggert, L., Fairhurst, G., and G. Shepherd, "UDP Usage Guidelines", BCP 145, RFC 8085, DOI 10.17487/RFC8085, March 2017, <<https://www.rfc-editor.org/info/rfc8085>>.
- [RFC9673] Hinden, R. and G. Fairhurst, "IPv6 Hop-by-Hop Options Processing Procedures", RFC 9673, DOI 10.17487/RFC9673, October 2024, <<https://www.rfc-editor.org/info/rfc9673>>.
- [I-D.ietf-ippm-asymmetrical-pkts] Mirsky, G., Ruffini, E., Nydell, H., Foote, R. F., and W.

Hawkins, "Performance Measurement with Asymmetrical Traffic Using Simple Two-Way Active Measurement Protocol (STAMP)", Work in Progress, Internet-Draft, draft-ietf-ippm-asymmetrical-pkts-14, 16 March 2026, <<https://datatracker.ietf.org/doc/html/draft-ietf-ippm-asymmetrical-pkts-14>>.

11.2. Informative References

- [RFC8250] Elkins, N., Hamilton, R., and M. Ackermann, "IPv6 Performance and Diagnostic Metrics (PDM) Destination Option", RFC 8250, DOI 10.17487/RFC8250, September 2017, <<https://www.rfc-editor.org/info/rfc8250>>.
- [RFC8754] Filsfils, C., Ed., Dukes, D., Ed., Previdi, S., Leddy, J., Matsushima, S., and D. Voyer, "IPv6 Segment Routing Header (SRH)", RFC 8754, DOI 10.17487/RFC8754, March 2020, <<https://www.rfc-editor.org/info/rfc8754>>.
- [RFC9197] Brockners, F., Ed., Bhandari, S., Ed., and T. Mizrahi, Ed., "Data Fields for In Situ Operations, Administration, and Maintenance (IOAM)", RFC 9197, DOI 10.17487/RFC9197, May 2022, <<https://www.rfc-editor.org/info/rfc9197>>.
- [RFC9268] Hinden, R. and G. Fairhurst, "IPv6 Minimum Path MTU Hop-by-Hop Option", RFC 9268, DOI 10.17487/RFC9268, August 2022, <<https://www.rfc-editor.org/info/rfc9268>>.
- [RFC9326] Song, H., Gafni, B., Brockners, F., Bhandari, S., and T. Mizrahi, "In Situ Operations, Administration, and Maintenance (IOAM) Direct Exporting", RFC 9326, DOI 10.17487/RFC9326, November 2022, <<https://www.rfc-editor.org/info/rfc9326>>.
- [RFC9343] Fioccola, G., Zhou, T., Cociglio, M., Qin, F., and R. Pang, "IPv6 Application of the Alternate-Marking Method", RFC 9343, DOI 10.17487/RFC9343, December 2022, <<https://www.rfc-editor.org/info/rfc9343>>.
- [RFC9486] Bhandari, S., Ed. and F. Brockners, Ed., "IPv6 Options for In Situ Operations, Administration, and Maintenance (IOAM)", RFC 9486, DOI 10.17487/RFC9486, September 2023, <<https://www.rfc-editor.org/info/rfc9486>>.
- [RFC7820] Mizrahi, T., "UDP Checksum Complement in the One-Way Active Measurement Protocol (OWAMP) and Two-Way Active Measurement Protocol (TWAMP)", RFC 7820, DOI 10.17487/RFC7820, March 2016, <<https://www.rfc-editor.org/info/rfc7820>>.
- [I-D.ietf-ippm-on-path-active-measurements] Fioccola, G., Zhu, K., Zhou, T., Zhu, Y., and X. Min, "On-Path Telemetry for Active Performance Measurements", Work in Progress, Internet-Draft, draft-ietf-ippm-on-path-active-measurements-03, 28 August 2026, <<https://datatracker.ietf.org/doc/html/draft-ietf-ippm-on-path-active-measurements-03>>.

[NIST-CSPRNG]

NIST Special Publication 800-90A, "Recommendation for Random Number Generation Using Deterministic Random Bit Generators", January 2012.

Acknowledgments

The authors would like to thank Greg Mirsky, Xiao Min, Tal Mizrahi, Cheng Li, Giuseppe Fioccola, Richard "Footer" Foote, and Jie Dong for reviewing this document and providing many useful comments and suggestions. The authors also thank William Hawkins for implementing the solution defined in this document in Teaparty. The authors would like to thank Vidhi Goel for the TSVART telechat review of [\[draftI-D.-ietf-mpls-stamp-pw\]](#), which resulted in the addition of the specifications for the IP/UDP header, UDP Checksum Handling, STAMP Session State, and Rate Limiting in this document. Thank you to Xiao Min for the PerfMetrdir review, and Marcus Ihlar for Shepherds review which helped improve this document.

Commenté [MB43]: Add as an informative reference.

Authors' Addresses

Rakesh Gandhi (editor)
Cisco Systems, Inc.
Canada
Email: rgandhi@cisco.com

Tianran Zhou
Huawei
China
Email: zhoutianran@huawei.com

Zhenqiang Li
China Mobile
China
Email: lizhenqiang@chinamobile.com

William Hawkins
University of Cincinnati
United States of America
Email: hawkinsw@obs.cr